Google Reviews Integration

Developer reference for the Carnex MERN project: files, functions, configuration and re-enable steps

Document date: 7 October 2026 • Applies to: branch `main` (after commit b16a70c)

Summary

The backend is still fully wired and live. The server already exposes `GET /api/google-reviews`, and it calls the Google Places API whenever `GOOGLE_PLACES_API_KEY` and `GOOGLE_PLACE_ID` are set. **Only the homepage UI is switched off.** It was commented out in `client/src/pages/public/Home.jsx` because billing for the Places API was not enabled yet. Turning the feature back on is mostly configuration plus uncommenting three blocks in one file.

- **Server (no code change needed):** service → controller → route → mounted in app.js.
- **Client (code change needed):** uncomment the imports, the state/effect and the JSX section in Home.jsx.
- **Configuration needed:** a Google Cloud project with billing, the Places API enabled, an API key and the business Place ID.
- **Important caveat:** the code uses the *legacy* Places API endpoint. Google Cloud projects created after March 2025 may not be able to enable the legacy API. If so, use the "Places API (New)" version of the service in Section 6.

1. Files related to Google Reviews

#	File	Layer	Status
1	server/src/services/googleReviewService.js	Server: Google API client + cache	Active
2	server/src/controllers/public/googleReviewController.js	Server: HTTP handler	Active
3	server/src/routes/public/googleReviews.js	Server: Express router	Active
4	server/app.js (lines 35 and 128)	Server: route mounting	Active
5	server/.env.example (lines 26-29)	Config template	Documented
6	client/src/services/reviewService.js (line 11)	Client: API call	Active (unused)
7	client/src/components/reviews/GoogleReviewCard.jsx	Client: single review card UI	Active (unused)
8	client/src/pages/public/Home.jsx	Client: homepage section	**Commented out**
9	client/src/components/ui/StarRating.jsx	Client: shared star display	Shared dependency

Not related to Google: these files belong to the separate in-house reviews system, which is customer-submitted reviews stored in MongoDB and moderated by an admin. Do not confuse them with the Google integration: `server/src/models/Review.js`, `controllers/public/reviewController.js`, `controllers/admin/reviewController.js`, `routes/public/reviews.js`, `routes/admin/reviews.js`, `validators/reviewValidators.js`, `client/.../ReviewCard.jsx`, `ReviewForm.jsx` and `pages/admin/AdminReviews.jsx`.

2. Function reference (file → function → responsibility)

2.1 server/src/services/googleReviewService.js

Symbol	Responsibility
CACHE_TTL_MS (line 6)	Cache lifetime: 6 hours. Change this to refresh reviews more or less often.
cache (line 7)	In-memory object { data, fetchedAt }. It is held per server process, so it is lost on restart and not shared between instances.
fetchGoogleReviews() (lines 14-73)	The **only** function that talks to Google. Steps: (a) reads the env vars and returns **null** if either is missing; (b) returns the cached data if it is still fresh; (c) calls the Place Details API, asking only for the fields *name, rating, user_ratings_total, url, reviews*; (d) throws if status is not OK; (e) normalises the response and sorts reviews newest first; (f) on an error it serves the stale cache if one exists, otherwise it rethrows.

Normalised object returned (this is the contract the frontend depends on):

```
{
  name:         string,   // business name
  rating:       number,   // average, e.g. 4.8
  totalRatings: number,   // user_ratings_total
  mapsUrl:      string,   // Google Maps listing URL
  reviews: [{
    authorName, authorPhoto, authorUrl,
    rating,          // 1-5
    text,
    relativeTime,    // "2 weeks ago"
    time             // unix seconds; used as the React key
  }]
}
```

2.2 server/src/controllers/public/googleReviewController.js

Function	Responsibility
getGoogleReviews(req, res, next)	Express handler for `GET /api/google-reviews`. It calls fetchGoogleReviews(). If that returns null (not configured), it responds **200** with `data: null` and the message "Google reviews are not configured yet". On success it responds 200 with the normalised object. On an error it logs the message and forwards ApiError.internal (500) to the error middleware.

2.3 server/src/routes/public/googleReviews.js and server/app.js

Location	Responsibility
googleReviews.js: router.get('/')	Maps the root path to getGoogleReviews.
app.js line 35	require('./src/routes/public/googleReviews')
app.js line 128	app.use(`${API_PREFIX}/google-reviews`, publicGoogleReviewRoutes): this is the public URL.

2.4 client/src/services/reviewService.js

Function	Responsibility
getGoogleReviews()	axios GET to `/google-reviews` through the shared api instance (baseURL = VITE_API_URL). The response payload is at res.data.data.

2.5 client/src/components/reviews/GoogleReviewCard.jsx

Component	Responsibility
GoogleLogo()	Inline SVG of the Google "G". It is required for attribution under the Google Maps Platform terms.
GoogleReviewCard({ review })	Renders a single review: the author photo (or an initial when there is no photo), the name, the relative time, the stars (via StarRating), the "n/5" score and the review text (clamped to 6 lines). It expects one item of the reviews[] array from 2.1.

2.6 client/src/pages/public/Home.jsx (currently commented out)

Block	Lines (approx.)	Responsibility
Imports	626-630	Imports getGoogleReviews and GoogleReviewCard.
GOOGLE_MAPS_URL const	638 (active)	The link used by the "Write a Review" button. It sends the user to the Google listing.
State + useEffect	783-796	Sets googleReviews / googleReviewsLoading and fetches the data once when the component mounts. On an error it sets null.
JSX section	1262-1313	Section header, aggregate rating line, "Write a Review" button, and a skeleton, grid or empty-state message.

*Line numbers are a snapshot. Search for the marker text **"GOOGLE REVIEWS — DISABLED"** to find the blocks reliably.*

3. End-to-end flow

```
Browser (Home.jsx mounts)
   |  useEffect -> getGoogleReviews()                [client/src/services/reviewService.js]
   v
GET {VITE_API_URL}/google-reviews
   |
   v
Express  app.use('/api/google-reviews', ...)          [server/app.js:128]
   |  router.get('/')                                 [server/src/routes/public/googleReviews.js]
   v
getGoogleReviews(req, res, next)                      [controllers/public/googleReviewController.js]
   |
   v
fetchGoogleReviews()                                  [services/googleReviewService.js]
   |-- env vars missing?  -> return null  -> 200 { data: null }
   |-- cache < 6h old?    -> return cache
   |-- axios GET maps.googleapis.com/maps/api/place/details/json
   |        ?place_id=...&fields=name,rating,user_ratings_total,url,reviews&key=...
   |-- status != OK       -> throw (or serve stale cache)
   |-- normalise + sort newest first -> store in cache -> return
   v
200 { success, data: { name, rating, totalRatings, mapsUrl, reviews[] } }
   |
   v
Home.jsx: setGoogleReviews(res.data.data)
   |-- loading         -> 3 x <SkeletonCard/>
   |-- reviews.length  -> grid of <GoogleReviewCard review={r}/>
   |-- otherwise       -> "Google reviews will appear here once connected."
```

The API key never reaches the browser. All calls to Google happen on the server, and the client only talks to your own API.

4. Required configuration and credentials

4.1 Environment variables (server only)

Variable	Where	Value
GOOGLE_PLACES_API_KEY	server/.env and the production host env settings	An API key from Google Cloud Console → APIs & Services → Credentials.
GOOGLE_PLACE_ID	server/.env and the production host env settings	The Place ID of "Carnex Auto Sales LLC" (it looks like ChIJ...). You can find it with Google's "Place ID Finder" tool.

Both variables are **optional**. They are not listed in `server/src/config/validateEnv.js`, so the server starts without them and the endpoint simply returns data: null. No client (VITE_) variables are needed beyond the existing VITE_API_URL.

4.2 Google Cloud setup checklist

- Create or select a Google Cloud project and **link a billing account**. This is mandatory, even if usage stays inside the free monthly credit.
- APIs & Services → Library → enable **"Places API"** (legacy, used by the current code). If only **"Places API (New)"** is offered, enable that and apply Section 6.
- Create an API key. **Restrict it:** under API restrictions, allow only the Places API; under Application restrictions, choose *IP addresses* and add the backend server's outbound IP. Do **not** use HTTP-referrer restrictions, because the key is used server-side.
- Get the Place ID from the Place ID Finder (search for the dealership name or address).

- Optional: set a quota or budget alert. With the 6-hour cache, each server instance makes at most about 4 calls per day.

> **Never commit real keys.** Put them only in server/.env (which is git-ignored) and in the hosting provider's environment settings. Keep the placeholders in .env.example.

5. Step-by-step: re-enable Google Reviews

Step 1: Configure the server

```
# server/.env
GOOGLE_PLACES_API_KEY=AIza...your_key
GOOGLE_PLACE_ID=ChIJ...your_place_id
```

Restart the backend. Environment variables are read on each request, but restarting also clears any cached state.

Step 2: Verify the backend before touching the UI

```
curl http://localhost:5000/api/google-reviews
```

Response	Meaning / fix
200, data: null, "not configured yet"	An env var is missing or misspelled, or the server was not restarted.
500 + server log "REQUEST_DENIED"	Billing is not enabled, the API is not enabled, or the key restriction blocks this server or IP.
500 + "INVALID_REQUEST" / "NOT_FOUND"	The Place ID is wrong.
500 + legacy API not enabled for the project	The project cannot use the legacy API. Apply Section 6.
200 with reviews[]	Working. Continue to step 3.

Step 3: Uncomment the UI in client/src/pages/public/Home.jsx

(a) **Imports** (around lines 626-630). Remove the "//" in front of the two imports:

```
import { getGoogleReviews } from '../../services/reviewService'
import { GoogleReviewCard } from '../../components/reviews/GoogleReviewCard'
```

(b) **State and effect** inside Home() (around lines 783-796). Uncomment this block:

```
const [googleReviews, setGoogleReviews] = useState(null)
const [googleReviewsLoading, setGoogleReviewsLoading] = useState(true)

useEffect(() => {
  getGoogleReviews()
    .then((res) => setGoogleReviews(res.data?.data || null))
    .catch(() => setGoogleReviews(null))
    .finally(() => setGoogleReviewsLoading(false))
}, [])
```

(c) **JSX section** (around lines 1262-1313). Delete the opening `{/*` placed just before `<section className="space-y-4">` and the closing `*/}` placed just after `</section>`. Keep the decorative banner comment above it, or update its wording.

No other changes are needed: SkeletonCard, Button, useState and useEffect are already imported in Home.jsx, and GOOGLE_MAPS_URL is already defined.

Step 4: Test

- Run the client and open the homepage. You should see a skeleton, then up to 5 review cards plus the aggregate rating.
- Check that profile photos load. The card uses referrerPolicy="no-referrer" for googleusercontent images.
- Check that "Write a Review" opens the Google listing (GOOGLE_MAPS_URL).
- Deploy: add both env vars on the production backend host, then redeploy the server and the client.

6. Optional: migrate to Places API (New)

Do this only if the legacy "Places API" cannot be enabled for your project. **Only one function changes:** `fetchGoogleReviews()` in `server/src/services/googleReviewService.js`. The returned shape stays the same, so the controller, route and every client file stay unchanged. In Google Cloud, enable **"Places API (New)"** and allow it in the key's API restrictions. Replace the try-block request and the normalisation with:

```
const { data } = await axios.get(
  `https://places.googleapis.com/v1/places/${encodeURIComponent(placeId)}`,
  {
    headers: {
      'X-Goog-Api-Key': apiKey,
      'X-Goog-FieldMask':
        'displayName,rating,userRatingCount,googleMapsUri,reviews'
    }
  }
)
// (New) API signals errors via HTTP status -> axios throws; no data.status check

const normalized = {
  name: data.displayName?.text,
  rating: data.rating,
  totalRatings: data.userRatingCount,
  mapsUrl: data.googleMapsUri,
  reviews: (data.reviews || [])
    .map((r) => ({
      authorName: r.authorAttribution?.displayName,
      authorPhoto: r.authorAttribution?.photoUri,
      authorUrl: r.authorAttribution?.uri,
      rating: r.rating,
      text: r.text?.text || r.originalText?.text || '',
      relativeTime: r.relativePublishTimeDescription,
      time: Math.floor(Date.parse(r.publishTime) / 1000)
    }))
    .sort((a, b) => b.time - a.time)
}
```

7. Where to make future changes (quick lookup)

I want to...	File	Function / place
Change how often reviews refresh	googleReviewService.js	CACHE_TTL_MS
Request extra Google fields	googleReviewService.js	fetchGoogleReviews(): the fields param and the normalized object
Show only 4-5 star reviews	googleReviewService.js	fetchGoogleReviews(): add .filter(r => r.rating >= 4) before .map
Get reviews in a specific language	googleReviewService.js	add language: 'en' to params (legacy) or ?languageCode=en (New)
Change the error or "not configured" response	googleReviewController.js	getGoogleReviews()
Change the URL path	server/app.js line 128 + reviewService.js	app.use(...) and getGoogleReviews()
Add rate limiting to the endpoint	server/app.js / routes/public/googleReviews.js	Attach a limiter from middleware/rateLimiter
Persist the cache across restarts or instances	googleReviewService.js	Replace the cache object with Redis or MongoDB (follow Google's caching terms)
Change the card look	GoogleReviewCard.jsx	GoogleReviewCard()

I want to...	File	Function / place
Change the section layout, heading or empty state	Home.jsx	the JSX section marked GOOGLE REVIEWS
Change the "Write a Review" link	Home.jsx	GOOGLE_MAPS_URL const
Show reviews on another page	that page	Copy the state/useEffect from Home.jsx and render GoogleReviewCard
Disable it again	Home.jsx	Re-comment the 3 blocks (the server can stay as is)

8. Known limitations

- **A maximum of 5 reviews** is returned by both the legacy API and the New API, and you cannot page through more. To display all reviews you need the Google Business Profile API, which requires OAuth as the business owner and access approval from Google. That is a larger change: new service, OAuth token storage and new env vars.
- Google chooses **which 5 reviews** it returns ("most relevant"). The code then sorts them newest first.
- The cache is in-memory and per instance. Every restart or cold start triggers one new API call.
- Google Maps Platform terms require Google attribution (the logo is already shown in GoogleReviewCard) and do not allow storing review content long term.
- If the API fails and there is no cache yet, the endpoint returns 500. The homepage then shows the empty-state message and nothing breaks.